RETAIL DATA AGENT · QUALITY ASSURANCE · AS OF 2026-11-01

DESIGN ONLY · NOT BUILT

How do we know the agent writes the right SQL?

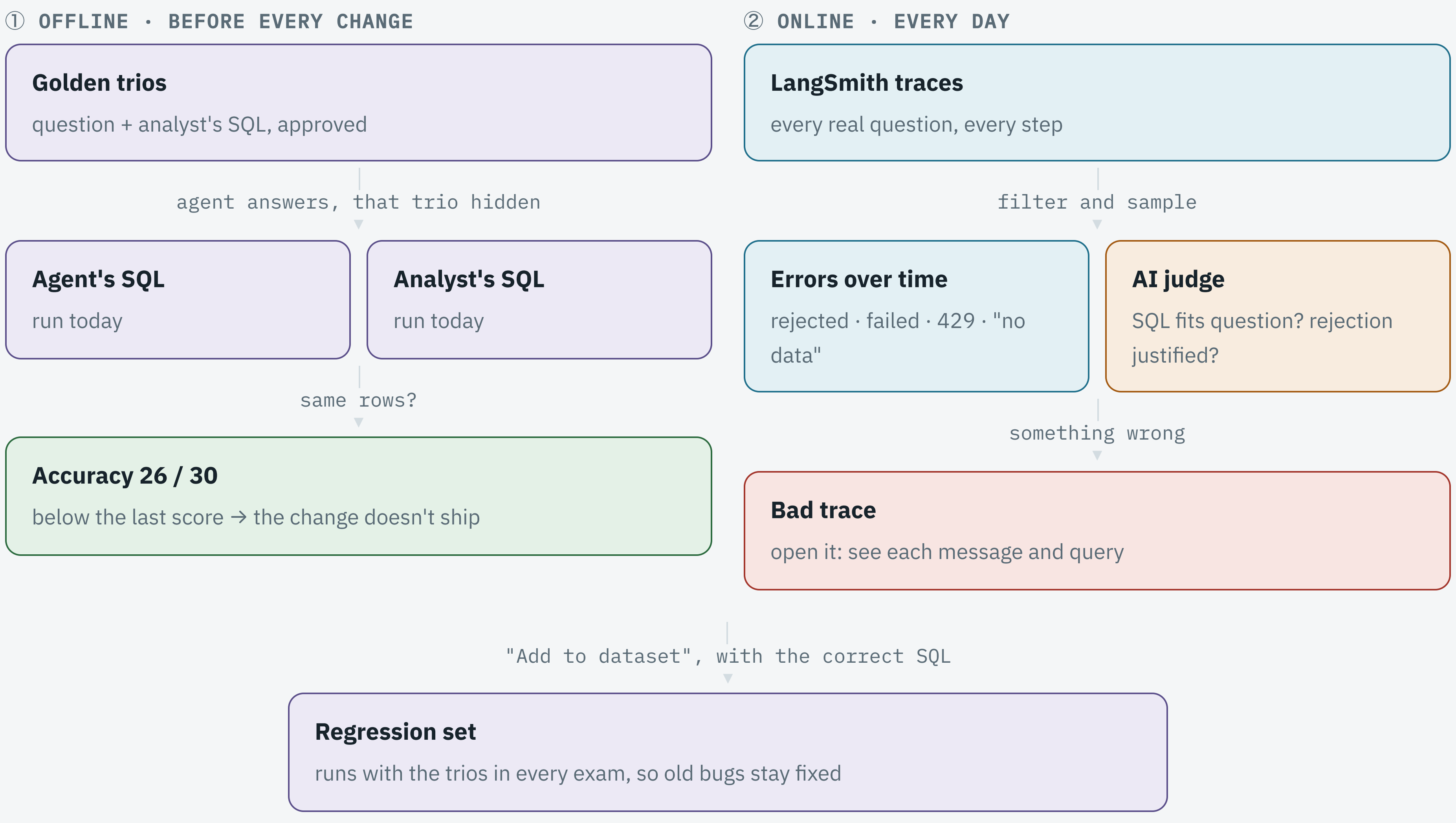
Two checks. Before any change ships, the agent sits an exam built from the analysts' Golden trios, and we count how often its SQL brings back the same rows. After it ships, we watch the LangSmith traces of real use for errors, and an AI judge reads a daily sample.
See also Golden Knowledge and user preferences.

🟪 offline: the exam 🟦 online: real traces 🟧 AI judge 🟩 pass 🟥 fail, block

In short

1. **The exam is the Golden trios.** Each analyst's question + SQL is a test case. The agent answers without seeing that trio.
2. **Compare rows, not SQL text,** on the same day. The data changes daily; the analyst's SQL doesn't go stale. Score: e.g. 26 of 30.
3. **Watch real use.** Traces show every rejected or failed query over time; an AI judge checks a daily sample: does the SQL fit the question?
4. **Every bad trace becomes a test,** so a fixed bug can't quietly come back.

The picture: an exam and a daily watch



① OFFLINE · BEFORE EVERY CHANGE

Golden trios
question + analyst's SQL, approved

agent answers, that trio hidden

Agent's SQL
run today

Analyst's SQL
run today

same rows?

Accuracy 26 / 30
below the last score → the change doesn't ship

② ONLINE · EVERY DAY

LangSmith traces
every real question, every step

filter and sample

Errors over time
rejected · failed · 429 · "no data"

AI judge
SQL fits question? rejection justified?

something wrong

Bad trace
open it: see each message and query

"Add to dataset", with the correct SQL

Regression set
runs with the trios in every exam, so old bugs stay fixed

🏠 Like a driving school: a final exam with known routes before the licence, then a dashcam on real drives. Every near-miss on the dashcam becomes a new exam route.

Example 1: the exam, one case

Question from a trio: **"Top 3 brands by revenue last quarter"**. The agent's SQL looks different from the analyst's, yet both bring back the same rows, so it passes. Comparing the text would have failed it.

```
-- agent (Golden trio hidden)
WHERE oi.created_at >= TIMESTAMP(DATE_SUB(
        DATE_TRUNC(CURRENT_DATE(), QUARTER),
        INTERVAL 1 QUARTER))
  AND oi.created_at < TIMESTAMP(
        DATE_TRUNC(CURRENT_DATE(), QUARTER))
GROUP BY p.brand
ORDER BY revenue DESC LIMIT 3
```

same rows?

```
-- analyst
WHERE DATE(oi.created_at) BETWEEN
        DATE_SUB(DATE_TRUNC(CURRENT_DATE(),
        QUARTER), INTERVAL 1 QUARTER)
  AND DATE_SUB(DATE_TRUNC(CURRENT_DATE(),
        QUARTER), INTERVAL 1 DAY)
GROUP BY brand
ORDER BY 2 DESC LIMIT 3
```

```
Diesel         20,508.87   20,508.87   ✅
True Religion  19,275.34   19,275.34   ✅
Calvin Klein   17,985.75   17,985.75   ✅    → case passes
```

The trio must be hidden from Golden retrieval during its own test, or the agent could copy the analyst's SQL and the score would be cheating.

Example 2: what the judge catches on live traces

"Revenue last quarter"

```
WHERE created_at >= TIMESTAMP_SUB(
  CURRENT_TIMESTAMP(), INTERVAL 90 DAY)
```

❌ Judge: "last quarter" means Jul 1 – Sep 30, but the SQL takes the last 90 days.

vs

A rejected query

```
SELECT city, COUNT(*) FROM users
GROUP BY city
```

Judge: the rejection was right only if it touched personal data. A safe query wrongly blocked is a **false alarm**, and it's counted too.

The judge is a different model from the agent, and it sees the question, the SQL and the tool results, not just the final text. Before we trust it, it also grades the exam cases: if it agrees with the row comparison on most of them (say 28 of 30), its live scores count.

The numbers we track

87%
exam accuracy (rows match), e.g. 26 of 30

1st try
right first time vs after a retry

% judged
live SQL that fits its question

👍 / 👎
user feedback, rephrased questions, answer time

SOURCE	WHAT WE LOOK AT	CHECKED BY
Golden trios	Agent's rows vs analyst's rows, same day	Code
Traces: errors	Share of questions with a rejected or failed SQL, a fallback or timeout, a "no data" answer; calls per question	Code, on a dashboard
Traces: sample	Does the SQL fit the question? Is every number in the answer from a tool result? Was each rejection justified?	AI judge (LangSmith evaluator)
Golden review	Candidate trios analysts rejected, counted by reason (wrong metric, period, join)	Analysts
Safety	No name or email in any answer; a brand manager's SQL always carries the brand filter; ≤ 5 calls	Code, pass or fail, never averaged

The first regression cases already exist

Bugs found while testing the prototype become the regression set's first entries, each with the answer it should have given:

QUESTION	WHAT WENT WRONG	SHOULD BE
Top 5 brands in Essen	"No sales" (every brand had under 5 buyers, so all rows were hidden)	"Hidden for privacy"
Nike's sales (as Levi's manager)	Off-topic reply after 4 queries	"I can only see Levi's", no SQL
Last-quarter report	`TIMESTAMP_SUB … MONTH` failed a whole round	DATE math, right first time
Q1 report action items	"Highest-converting traffic sources" with no query behind it	Every claim backed by a number

> **The rule in one sentence**
>
> No change ships unless the agent, without seeing the answers, still gets at least as many Golden and regression questions right as before, counted by comparing rows, not words.

Status: design only (brief requirement 6, with 7 for the online part). Design: "Evaluation" in `docs/hld.md`. Built today: 185 unit tests (`uv run pytest`) with a scripted model and a fake database, covering the gateway's safety rules, not answer quality.

Would plug in as: a script that loads `golden.trios` plus the LangSmith regression dataset, runs the agent with that trio excluded from `{golden-examples}`, executes both SQLs through the same gateway, and compares result sets (order ignored unless ranked, numbers within rounding). Runs on a paid key: the free tier allows about 20 calls a day per model.

Online: LangSmith dashboards over trace metadata (`rewritten_sql`, rows, bytes, errors, model) and an LLM-as-judge evaluator on a daily sample; user 👍/👎 stored as LangSmith feedback.

See also Golden Knowledge, user preferences, SQL errors, small groups, architecture.